Which checks would challenge our assumptions rather than merely repeat them?

Student notes for lecture 8. Find the boundary a friendly example hides, group inputs and test their edges, match each check to what it can observe, explore a journey, derive expected results independently, state properties and reduce a counterexample, compare executed code with checked code, control the clock, and let a check guide a change.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	Sixty minutes passes. Is the rule right?	Anchor	`week-08.html#/friendly-example`
02	Do you have to test every minute?	Bridge	`week-08.html#/partition-the-inputs`
03	The overlap function passes. Can two people still get one room?	Anchor	`week-08.html#/choose-test-scope`
04	Who checked the expected answer?	Bridge	`week-08.html#/oracle-independence`
05	What must stay true after any accepted booking?	Anchor	`week-08.html#/properties-and-counterexamples`
06	The line ran. Did anything check its answer?	Bridge	`week-08.html#/coverage-and-mutation`
07	Same code, same check. Why does it pass and then fail?	Bridge	`week-08.html#/repeatable-observation`
08	What should happen before you fix the bug?	Bridge	`week-08.html#/testing-as-feedback`

Campus Rooms is a fictional teaching example, and its clock readings, delays and test runs are simulated. The 90-minute trial is temporary; the canonical 30-to-120-minute inclusive range stays in force. Code is Python-like pseudocode.

The check is green. Why was Ari’s booking refused?

The rule shipped with a passing check, and every line ran. Then Ari asked for the longest booking it allows.

R-02: A booking lasts 30 through 120 minutes, both included. The only check was “assert duration_ok(60) == True”.

THE RUN

1 of 1 checks passed. Lines executed: 2 of 2.

WHAT ARI SEES

C101, 10:00 to 12:00, 120 minutes. The screen says “Choose 30 to 120 minutes” and answers Rejected.

Prediction: the check is green, so will the 120-minute booking go through? It is refused. The check told the truth about 60 minutes and was never asked about 120.

> The rest of this lecture asks what a check can tell you, and which checks would have challenged this code.

You have the rules. How good is the evidence behind them?

Week 2 linked requirements to checks. Week 7 reviewed them. Now the checks themselves are under examination.

What you will be able to explain

- Derive ordinary, boundary, and invalid cases from a stated rule.
- Choose a suitable scope for a check and explain what it leaves unexamined.
- Distinguish executing code from asserting the intended result.
- Explain a property and a small mutation-based challenge to a test suite.
- Recognize shared assumptions, uncontrolled inputs, and flaky observations.

The rules under test

- R-01: No overlapping confirmed bookings for the same room. [10:00,11:00) and [11:00,12:00) are adjacent, and adjacent is allowed.
- R-02: A booking lasts 30 through 120 minutes, both included.
- R-03: You can cancel your own future confirmed booking.
- R-04: A failed notification leaves a confirmed booking confirmed.

How to read a check

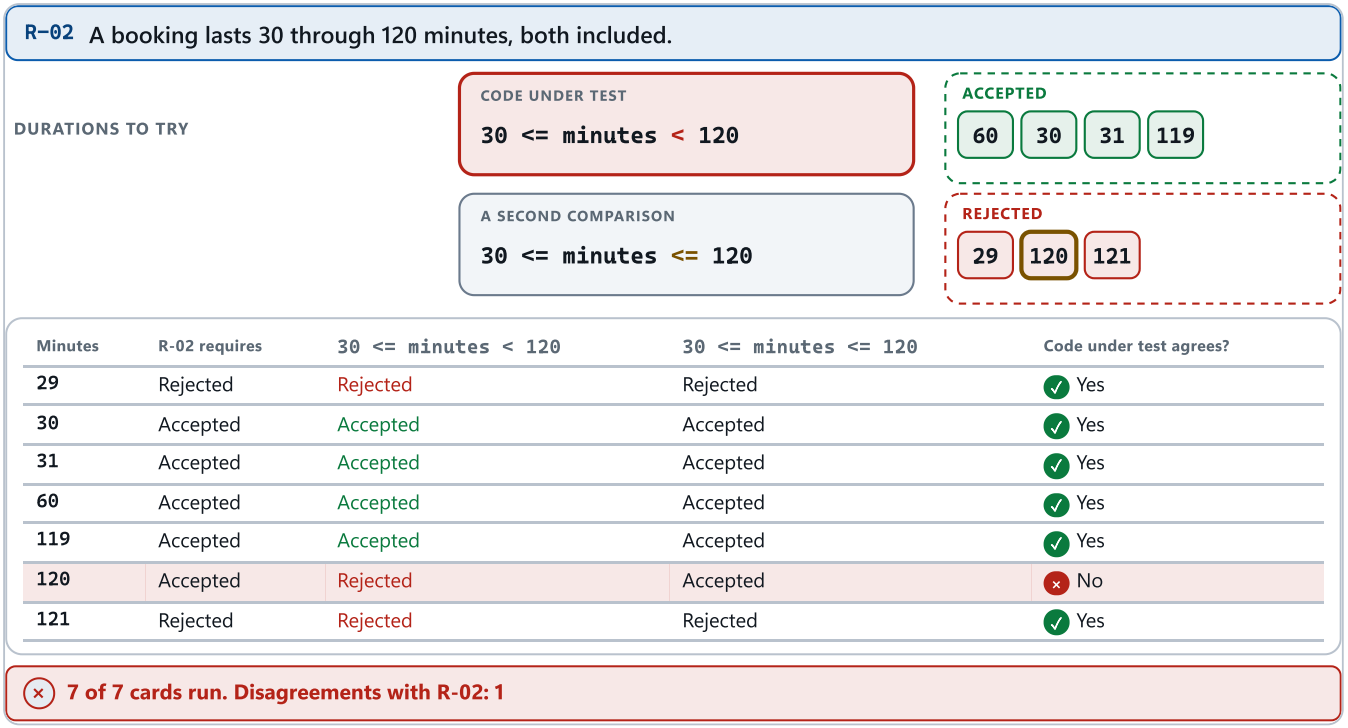
assert duration_ok(60) == True. Call the function with 60. The answer must be True. If it is anything else, the check fails.

Sixty minutes passes. Is the rule right?

Panel 1 of 3 · The code as shipped: one row disagrees

Condition: **The code as shipped: the upper comparison is <** State: **8 of 8**

A 60-minute request passes under both comparisons, so that example cannot tell them apart. At 120, the shipped comparison answers Rejected while R-02 says Accepted.



Minutes	R-02 requires	`30 <= minutes < 120`	`30 <= minutes <= 120`	Code under test agrees?
29	Rejected	Rejected	Rejected	✓ Yes
30	Accepted	Accepted	Accepted	✓ Yes
31	Accepted	Accepted	Accepted	✓ Yes
60	Accepted	Accepted	Accepted	✓ Yes
119	Accepted	Accepted	Accepted	✓ Yes
120	Accepted	Rejected	Accepted	✗ No
121	Rejected	Rejected	Rejected	✓ Yes

7 of 7 cards run. Disagreements with R-02: 1

The rule is 30 through 120 minutes, both included. The shipped predicate is 30 <= minutes < 120; the correct predicate is 30 <= minutes <= 120. The table records the friendly example and the six boundary cases.

Principle · Test cases should distinguish plausible incorrect behavior from the intended behavior.

In short · Pick the case that can tell right from nearly right.

Sixty minutes passes. Is the rule right?

Panel 2 of 3 · Changed: the inclusive comparison

Condition: **Only the comparison changes: the upper comparison is <=** State: **4 of 4**

121 is still rejected. The fix restored the endpoint and kept the limit.

R-02 A booking lasts 30 through 120 minutes, both included.

DURATIONS TO TRY

CODE UNDER TEST

```
30 <= minutes <= 120
```

A SECOND COMPARISON

```
30 <= minutes < 120
```

ACCEPTED

60 30 31 119 120

REJECTED

29 121

Minutes	R-02 requires	30 <= minutes <= 120	30 <= minutes < 120	Code under test agrees?
29	Rejected	Rejected	Rejected	✓ Yes
30	Accepted	Accepted	Accepted	✓ Yes
31	Accepted	Accepted	Accepted	✓ Yes
60	Accepted	Accepted	Accepted	✓ Yes
119	Accepted	Accepted	Accepted	✓ Yes
120	Accepted	Accepted	Rejected	✓ Yes
121	Rejected	Rejected	Rejected	✓ Yes

✓ **7 of 7 cards run. Disagreements with R-02: 0**

Replaying the same cases, 120 is accepted and 121 is still rejected. The fix restored the endpoint and kept the limit.

Principle · Test cases should distinguish plausible incorrect behavior from the intended behavior.

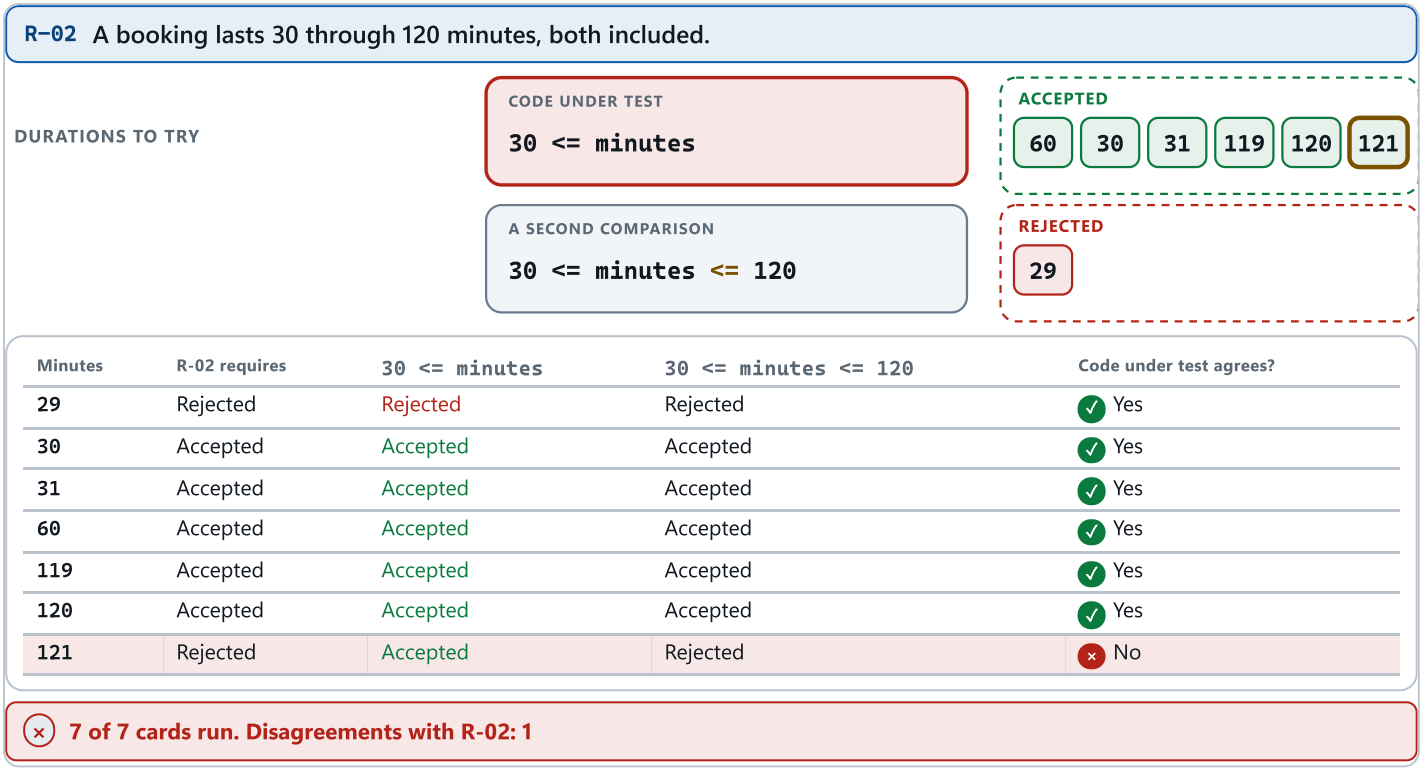
In short · Pick the case that can tell right from nearly right.

Sixty minutes passes. Is the rule right?

Panel 3 of 3 · Changed: a repair that goes too far

Condition: **A repair that goes too far: the upper comparison is removed** State: **4 of 4**

121 is accepted, and R-02 says rejected. 120 and 121 together separate an inclusive limit from no limit at all.



Minutes	R-02 requires	`30 <= minutes`	`30 <= minutes <= 120`	Code under test agrees?
29	Rejected	Rejected	Rejected	✓ Yes
30	Accepted	Accepted	Accepted	✓ Yes
31	Accepted	Accepted	Accepted	✓ Yes
60	Accepted	Accepted	Accepted	✓ Yes
119	Accepted	Accepted	Accepted	✓ Yes
120	Accepted	Accepted	Accepted	✓ Yes
121	Rejected	Accepted	Rejected	✗ No

⊗ **7 of 7 cards run. Disagreements with R-02: 1**

With no upper comparison, 120 is accepted and so is 121. The two cases together distinguish an inclusive 120-minute limit from no upper limit, under otherwise valid conditions.

Principle · Test cases should distinguish plausible incorrect behavior from the intended behavior.

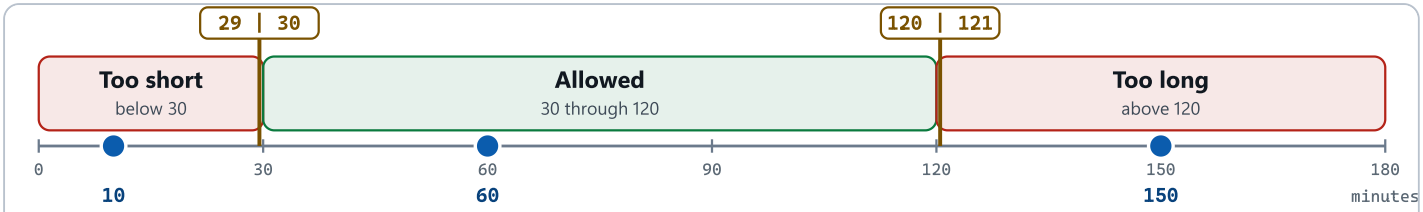
In short · Pick the case that can tell right from nearly right.

Do you have to test every minute?

Panel 1 of 2 · Three groups, two edges and two more concerns

Condition: **R-02 as written: 30 through 120 minutes** State: **6 of 6**

Each group keeps its own expected answer and reason. The edges show where one answer becomes the next.



ONE VALUE FROM EACH GROUP

Minutes	R-02 requires	`30 <= minutes < 120`
`10`	Rejected	Rejected
`60`	Accepted	Accepted
`150`	Rejected	Rejected

WHERE THE ANSWER CHANGES

`29 | 30` Rejected becomes Accepted

`120 | 121` Accepted becomes Rejected

Just inside each edge: 31 and 119.

TWO CONCERNS: HOW LONG, AND IS IT A VALID REQUEST AT ALL

Request	Too short · 29	Allowed · 60	Too long · 121
Start before end `[10:00,11:00)`	Rejected: duration	Accepted	Rejected: duration
End before start `[11:00,10:00)`	**Rejected: order**. End must be later than start. There is no duration to judge.		
Not whole minutes `45.5 · "soon" · missing`	**Rejected: shape**. The value is not a whole number of minutes.		

Too short, allowed and too long each share one expected answer, and the decision changes between 29 and 30 and between 120 and 121. One value per group (10, 60, 150) passes even the code that leaves out 120, so the edge cases are needed as well. Reversed and malformed requests are refused for different reasons.

Principle · Partitions organize expectations; boundaries expose where one expectation becomes another.

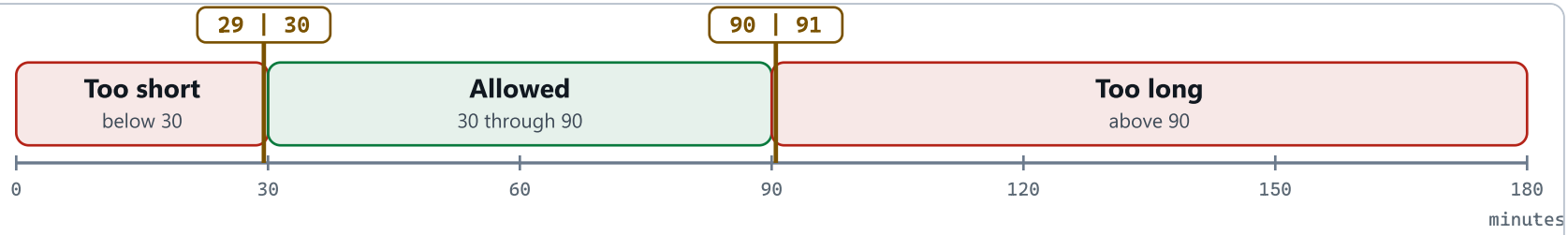
In short · Group the same answers, then test the edges.

Do you have to test every minute?

Panel 2 of 2 · Changed: a labeled 90-minute trial

Condition: **Temporary trial: the upper limit is 90 minutes** State: **3 of 4**

Read the table. The lower cases stay. **91, 119 and 120 change their answer**, and **89, 90 and 91** become the new edge cases.



Temporary trial Temporary trial: the upper limit is 90 minutes

Minutes	R-02: up to 120	Trial: up to 90	What the case does now
29	Rejected	Rejected	Same answer, same job
30	Accepted	Accepted	Same answer, same job
31	Accepted	Accepted	Same answer, same job
89	Accepted	Accepted	New: just inside the new edge
90	Accepted	Accepted	New: the last allowed value
91	Accepted	Rejected	New: the first refused value
119	Accepted	Rejected	Answer changes; no longer near an edge
120	Accepted	Rejected	Answer changes; no longer the edge
121	Rejected	Rejected	Same answer; no longer the edge

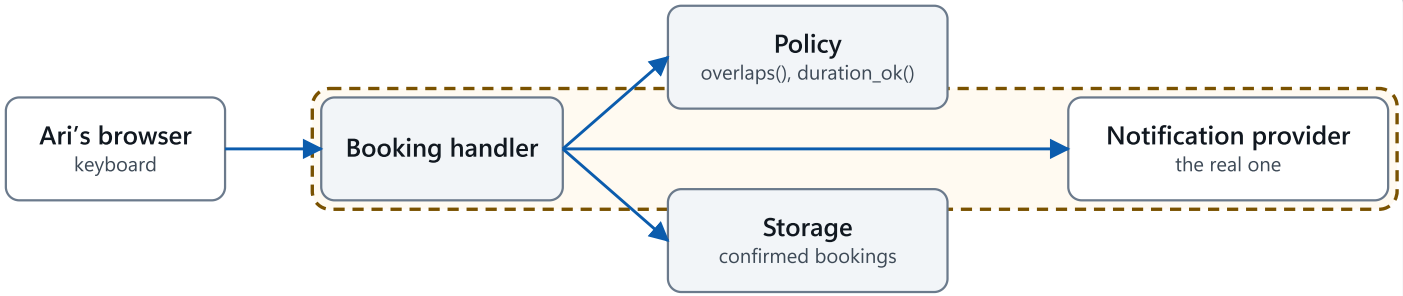
> Compare each case under the two rules. The lower edge cases keep their job. 91, 119 and 120 change their answer, and 89, 90 and 91 become the new edge cases. The trial is temporary; R-02 returns to 120 inclusive.

The overlap function passes. Can two people still get one room?

Panel 1 of 3 · A claim-to-scope matrix with one open limit per row

Condition: **The claims and the real boundaries** State: **6 of 6**

Each row pairs a claim with the place it can be observed, and names what stays open. Choose from the claim and the risk of being wrong.



✓ **Five claims, five places to look. The notification boundary comes next**

What you want to know	Check scope	What it leaves open
the interval predicate is correct	unit: pure function	says nothing about concurrent storage
the handler rejects conflicts using Policy	component with a real store, one request at a time	no interleavings
no double booking under concurrency	storage-boundary concurrent check	covers the tested interleavings only
a user completes a booking by keyboard and sees the status	end-to-end journey	slow; few cases
notification failure does not undo confirmation	integration with a controlled double plus a contract check against the real provider	the double encodes assumptions about the provider

A unit check of the interval predicate says nothing about concurrent storage. The concurrent check at the storage boundary is the one that can observe double booking, and it covers the interleavings it tried. The journey is slow and covers few cases.

Principle · The appropriate test scope follows the claim and failure risk, not the prestige of a test label.

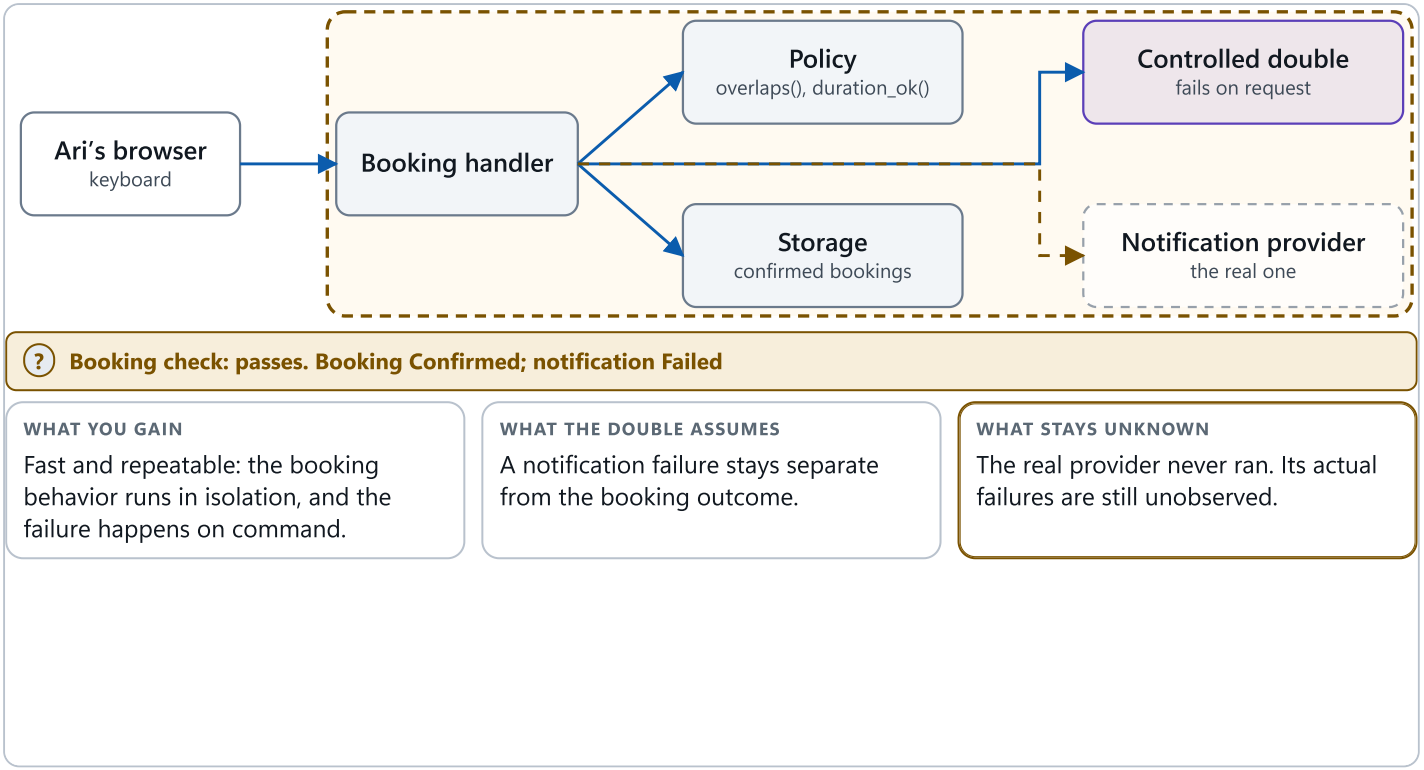
In short · Choose the check by what you need to know.

The overlap function passes. Can two people still get one room?

Panel 2 of 3 · Changed: a controlled double replaces the provider

Condition: **The real notification boundary is replaced by a controlled double** State: **4 of 4**

The real provider never ran. A double returning the answer you wanted tells you how booking behaves **if the provider behaves as assumed**.



The double makes the failure repeatable and isolates the booking behavior. It encodes an assumption about the provider, and the real provider did not run. A mock returning the desired value leaves that question open.

Principle · The appropriate test scope follows the claim and failure risk, not the prestige of a test label.

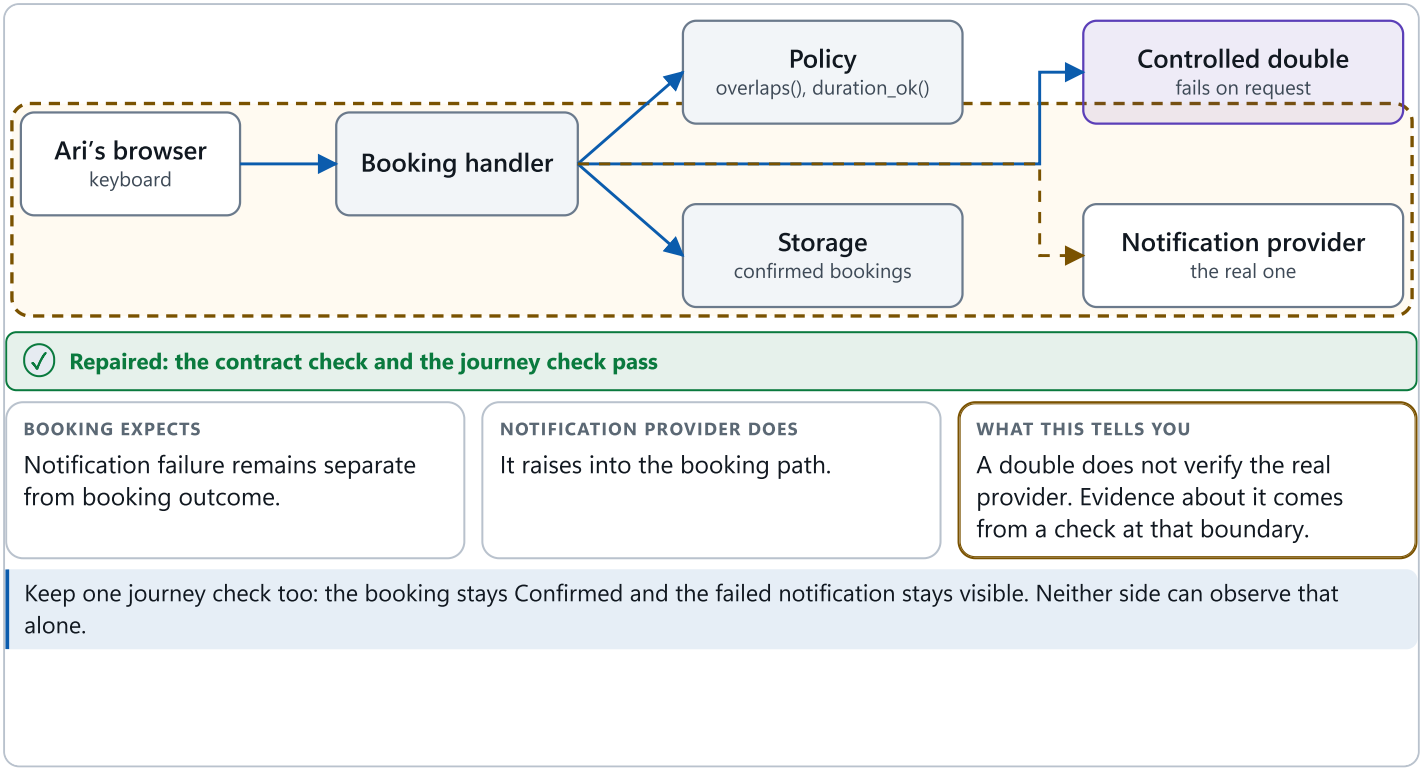
In short · Choose the check by what you need to know.

The overlap function passes. Can two people still get one room?

Panel 3 of 3 · Changed: the actual provider violates the expectation

Condition: **The actual provider violates what the consumer expects** State: **4 of 4**

Repair the boundary, and keep one journey check for what neither side can observe alone.



Booking expects a notification failure to remain separate from the booking outcome; the provider raises into the booking path. A contract or integration check at the real boundary shows the mismatch. One journey check stays for the behavior neither side can establish alone.

Principle · The appropriate test scope follows the claim and failure risk, not the prestige of a test label.

In short · Choose the check by what you need to know.

What do you check when you do not know what to check?

Start from one question about recovery. Let each observation choose the next check.

The question: How does someone recover from a lost response? This is a simulated example, not a report of a study.

Each observation chooses the next check

- Submit by keyboard: The screen says Submitting. The response never arrives. What can Ari do now?
- Reopen the booking: The stored booking comes back Confirmed, and the screen still says Submitting. Is the mismatch in the storage, or in the screen?
- Check the correction: After the correction, reopening shows Confirmed. Which check keeps this from coming back?
- Keep it as a check: The discovery is now a repeatable regression example. What else would this journey reveal?

> Found: retrieved confirmation and the displayed status disagree. The screen said Submitting while the stored booking was confirmed; after the correction it says Confirmed.

The resulting regression example

```
reopen(booking)
assert shown_status() == "Confirmed"
```

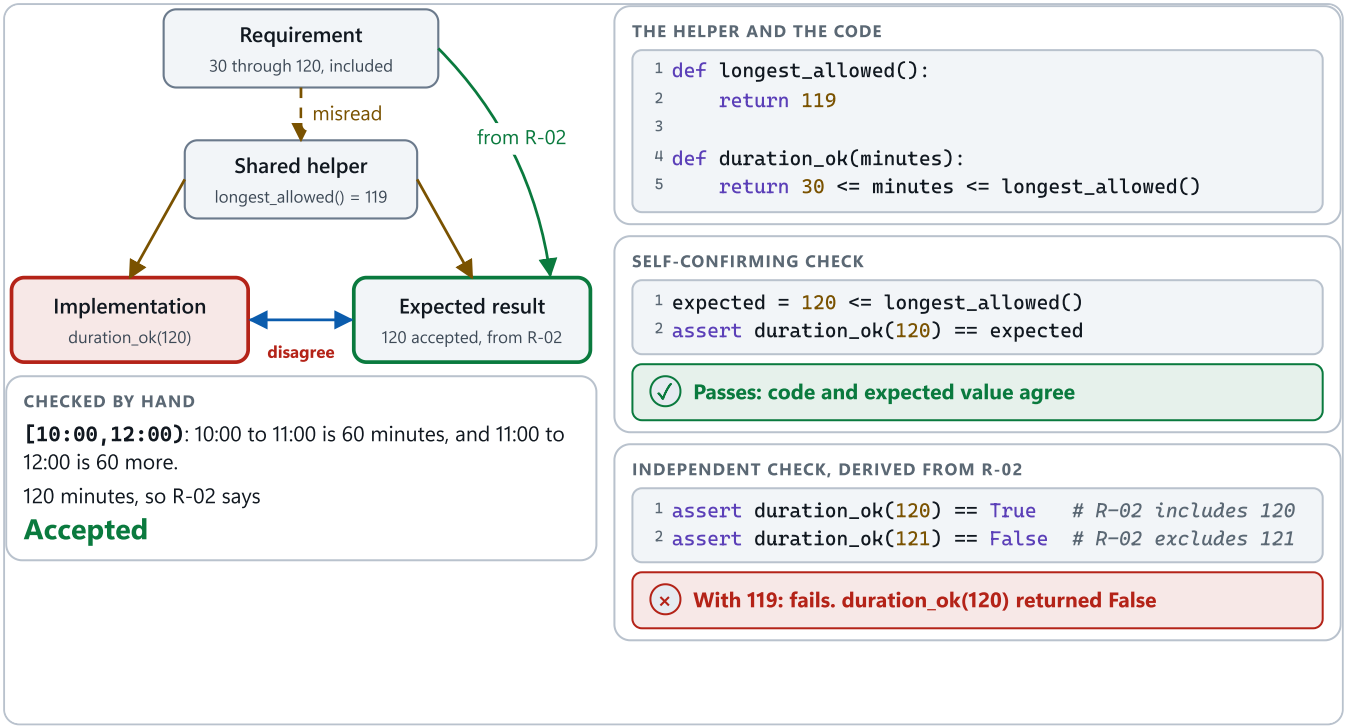
Scripted checks keep known expectations. Exploration finds new ones. A usability study asks a separate question: Can a representative person complete recovery and explain the outcome?

Who checked the expected answer?

Panel 1 of 3 · Where each expected result comes from

Condition: **The code and the expected value read one helper** State: **5 of 6**

Check it by hand. In **[10:00,12:00)**, 10:00 to 11:00 is 60 minutes, and 11:00 to 12:00 is 60 more, so 120 minutes. R-02 accepts that. **The independent check fails and exposes the defect.**



The self-confirming check computes its expected value with the same helper the code uses, so it passes while the code refuses 120. The independent expectation comes from R-02: [10:00,12:00) is 60 minutes and 60 more, 120 in all, and R-02 accepts it. That check fails and exposes the defect.

Principle · A test oracle—the basis for the expected result—must not simply repeat the defect being examined.

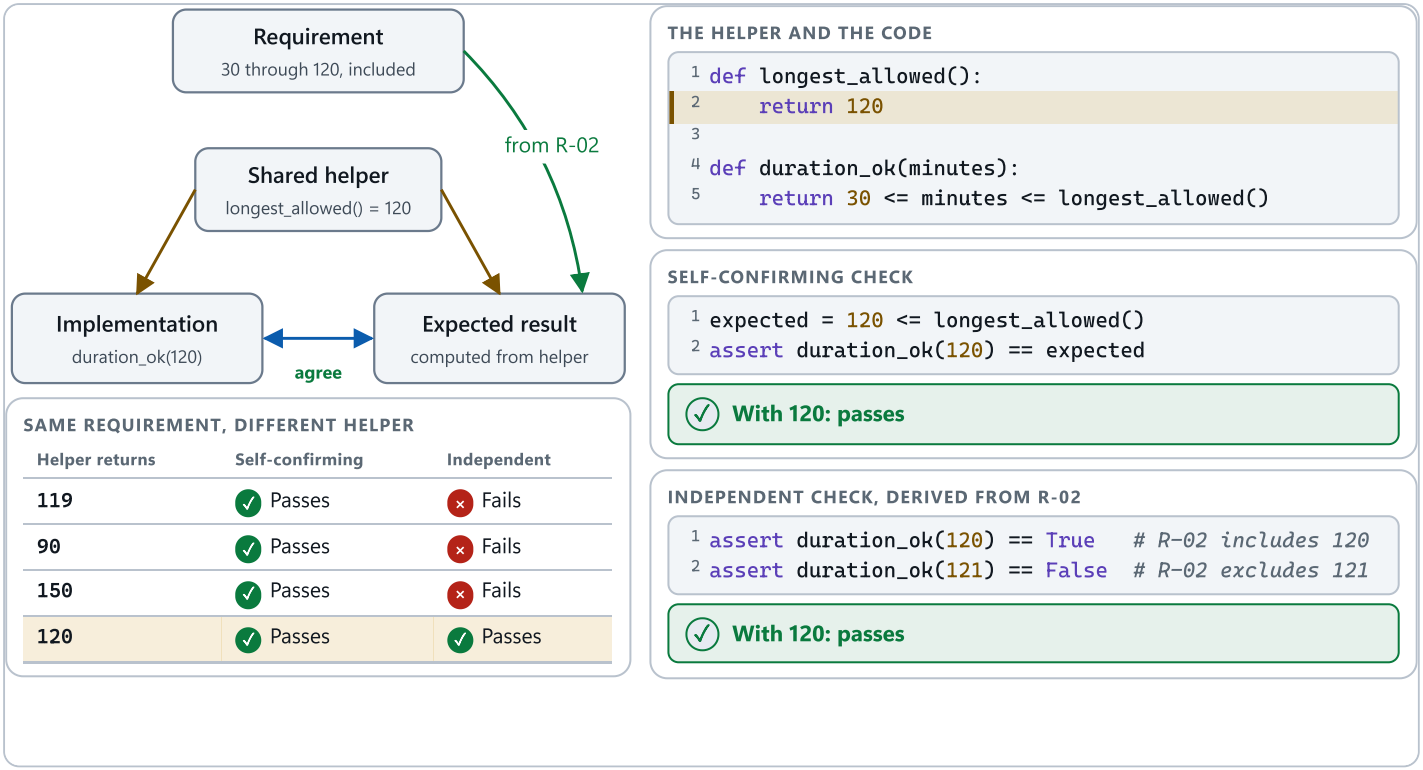
In short · Get the expected answer from the requirement, not from the code.

Who checked the expected answer?

Panel 2 of 3 · Changed: the helper moves, and the requirement stays

Condition: **The helper changes while the requirement stays fixed** State: **4 of 4**

Helper at **120**: now both pass. Only the independent check could tell this value from the others.



For helper values 119, 90 and 150 the self-confirming check passes and the independent check fails. At 120 both pass. A check that passes for every value of the helper tells you little about that value.

Principle · A test oracle—the basis for the expected result—must not simply repeat the defect being examined.

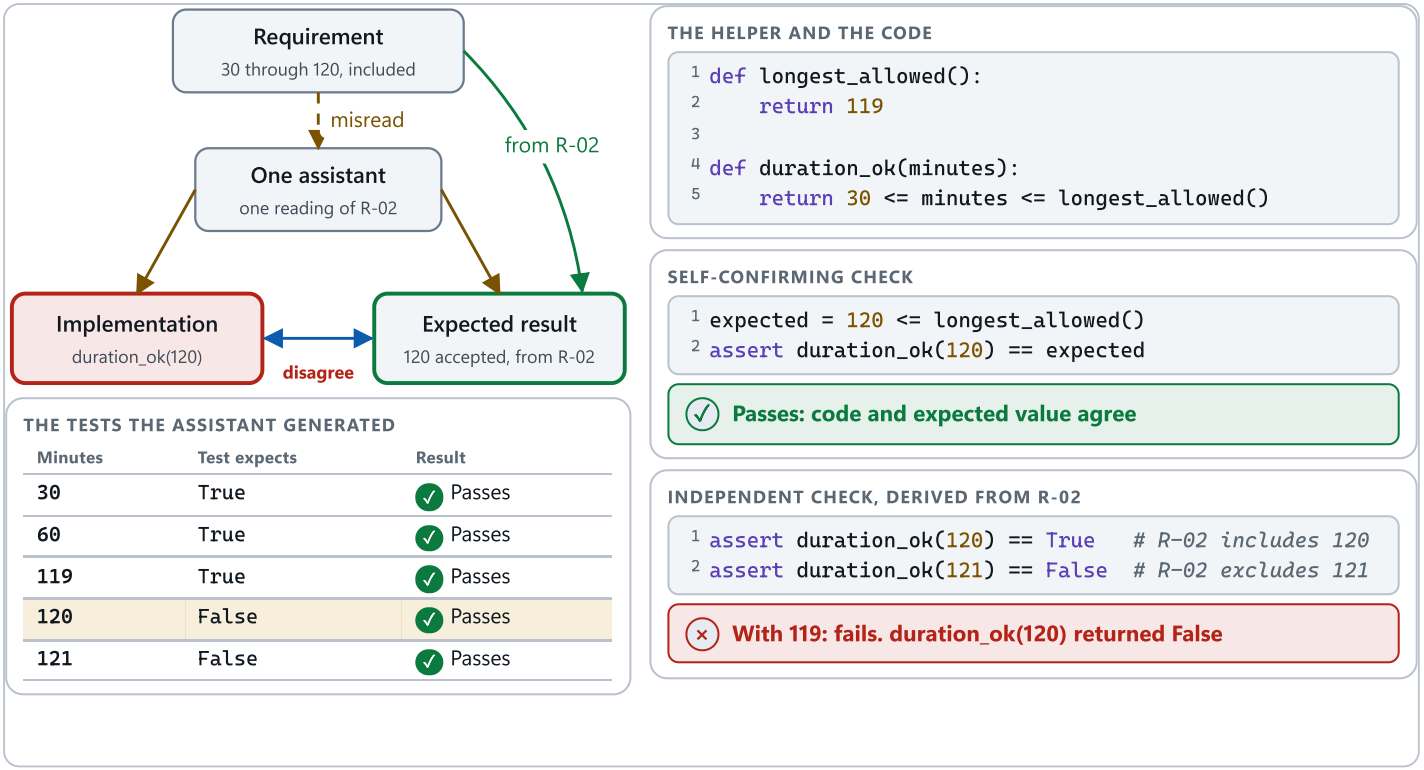
In short · Get the expected answer from the requirement, not from the code.

Who checked the expected answer?

Panel 3 of 3 · Changed: one assistant drafted the code and the tests

Condition: **One assistant drafted the helper and its tests** State: **3 of 3**

An independent check, **derived from R-02 as written**, expects 120 to be accepted. Against this candidate it fails. Week 13 returns to this loop.



All five generated tests pass, and the 120 test expects False: it shares the misreading. A check derived from R-02 as written fails against this candidate. Week 13 returns to this loop.

Principle · A test oracle—the basis for the expected result—must not simply repeat the defect being examined.

In short · Get the expected answer from the requirement, not from the code.

What must stay true after any accepted booking?

Panel 1 of 4 · Two properties and a replayable sequence

Condition: **Strict overlap: touching intervals are allowed** State: **6 of 6**

A property-based tool generates many sequences and tests these same statements. These four are fixed, so you can replay them.

P1 No two confirmed bookings for the same room overlap — Holds

P2 A valid adjacent booking is accepted — Holds

overlaps(a, b): `a.start < b.end and b.start < a.end`



Room	Candidate	Result	Why
C101	[10:00,11:00)	Accepted	The room is free
C101	[10:30,11:30)	Rejected	Overlaps [10:00,11:00)
C101	[11:00,12:00)	Accepted	Adjacent: its start meets the earlier end
C202	[10:30,11:30)	Accepted	A different room

P1: no two confirmed bookings for the same room overlap. P2: a valid adjacent booking is accepted. Intervals are half-open, so [10:00,11:00) and [11:00,12:00) are adjacent and allowed. These four candidates are fixed examples; property-based tools generate many more.

Principle · Properties expose classes of behavior, but the properties themselves must express the intended rule.

In short · State what must always hold, and check the statement too.

What must stay true after any accepted booking?

Panel 2 of 4 · Changed: the comparison rejects adjacency

Condition: **The comparison also rejects adjacent bookings** State: **4 of 4**

A service that refuses everything satisfies P1. You need P2 as well: the properties have to say the whole rule.

P1	No two confirmed bookings for the same room overlap	Holds
P2	A valid adjacent booking is accepted	Fails

overlaps(a, b): `a.start <= b.end and b.start <= a.end`



Room	Candidate	Result	Why
C101	[10:00,11:00)	Accepted	The room is free
C101	[11:00,12:00)	Rejected	Overlap, says the comparison; adjacent, allowed, says the rule

Smallest displayed counterexample: [10:00,11:00) then [11:00,12:00). The comparison calls it an overlap and the rule calls it adjacent. P1 holds and P2 fails. A service that refuses everything satisfies P1, which is why both properties are needed.

Principle · Properties expose classes of behavior, but the properties themselves must express the intended rule.

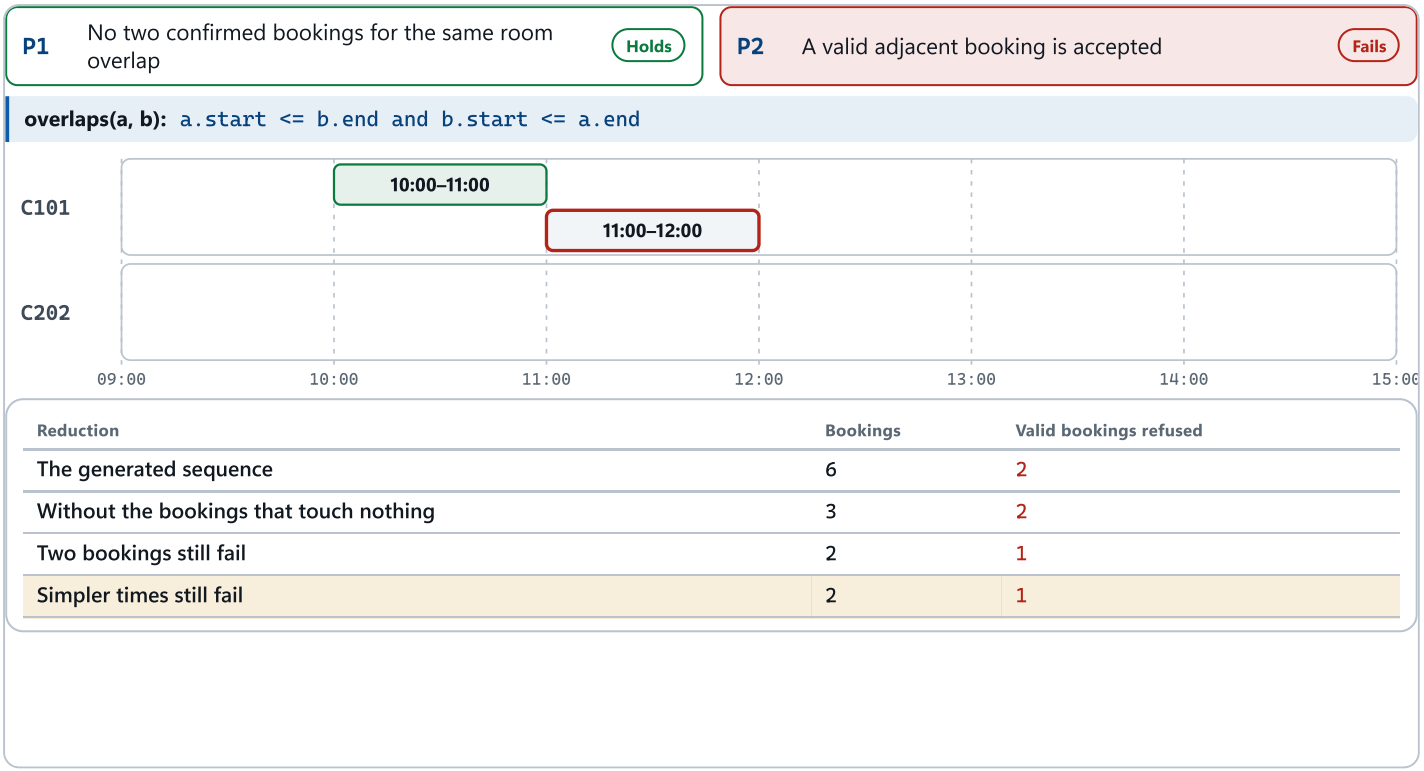
In short · State what must always hold, and check the statement too.

What must stay true after any accepted booking?

Panel 3 of 4 · A generated failure reduced to its smallest form

Condition: **A generated failure, reduced to its smallest form** State: **4 of 4**

Simpler times still fail. This is the smallest counterexample: [10:00,11:00) then [11:00,12:00). Tools call this reduction shrinking.



Reduction	Bookings	Valid bookings refused
The generated sequence	6	2
Without the bookings that touch nothing	3	2
Two bookings still fail	2	1
Simpler times still fail	2	1

Six generated bookings reduce to three, then two, then to simpler times, and P2 fails at every step. Property-based tools report this smaller counterexample so you can see the cause.

Principle · Properties expose classes of behavior, but the properties themselves must express the intended rule.

In short · State what must always hold, and check the statement too.

What must stay true after any accepted booking?

Panel 4 of 4 · Optional depth: fuzzing with malformed text

Condition: **Malformed text instead of a valid duration** State: **4 of 4**

After the correction, nothing crashes. Blank text gets a clear answer: “Rejected: enter a whole number of minutes.”

P	Any text gets a clear answer, and nothing crashes.	Holds
`"90"`	a number, written as text	Clear answer
`"ninety"`	a word	Clear answer
`"  "`	three spaces	Clear answer
`" "`	one space	Clear answer
`""`	empty text	Clear answer

Fuzzing keeps feeding generated input. The property stays simple: any text gets a clear answer, and nothing crashes.

A fuzzer sends generated input, and the property is that any text gets a clear answer without a crash. The failing input reduces from three spaces to empty text. After the correction, blank text gets a clear answer and nothing crashes.

Principle · Properties expose classes of behavior, but the properties themselves must express the intended rule.

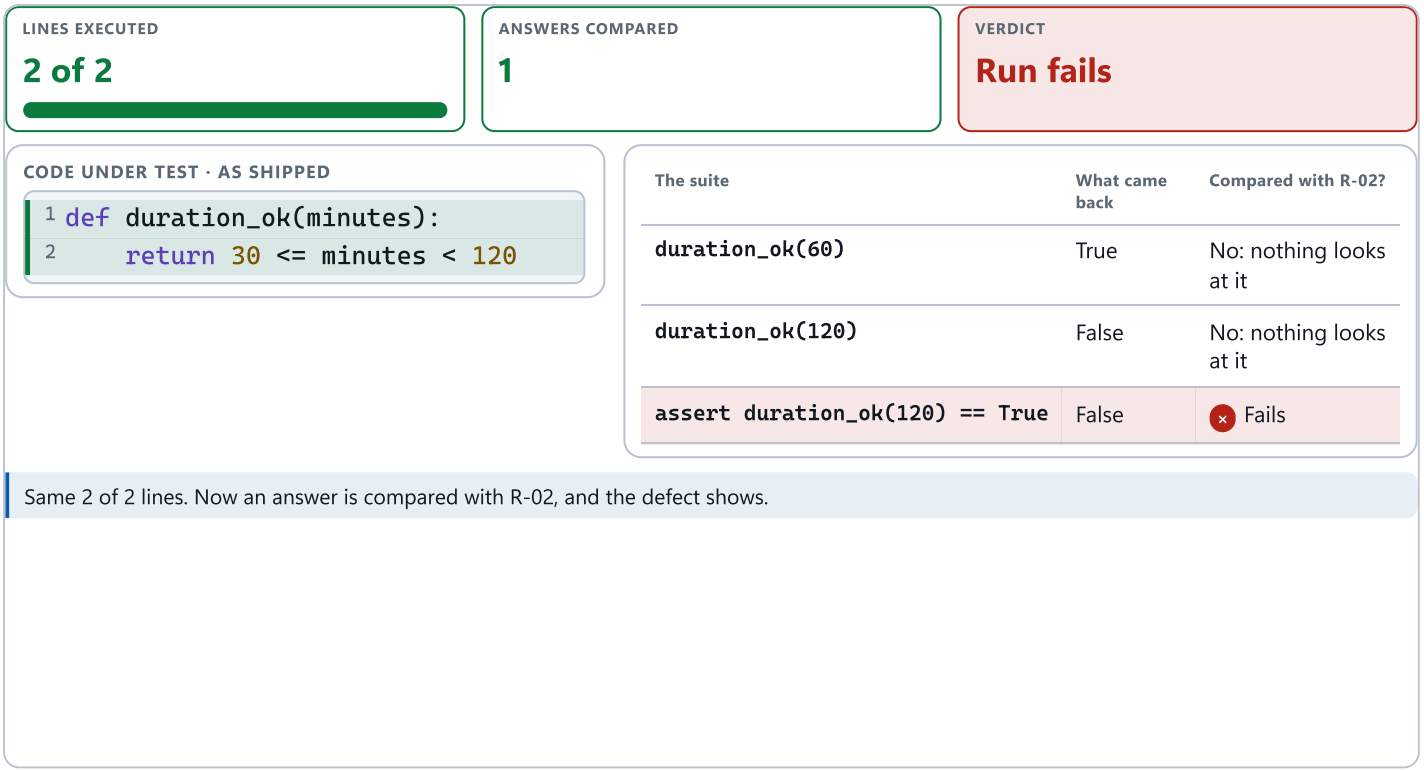
In short · State what must always hold, and check the statement too.

The line ran. Did anything check its answer?

Panel 1 of 4 · The line ran, and the answer went unread

Condition: **The shipped code and a suite with no assertions** State: **4 of 4**

Add one assertion. **Coverage stays at 2 of 2, and the run now fails.** The same lines ran; the evidence is different.



The suite called duration_ok(120) and never looked at the answer. Lines executed: 2 of 2. One added assertion leaves coverage the same and makes the run fail, because the shipped comparison refuses 120.

Principle · Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

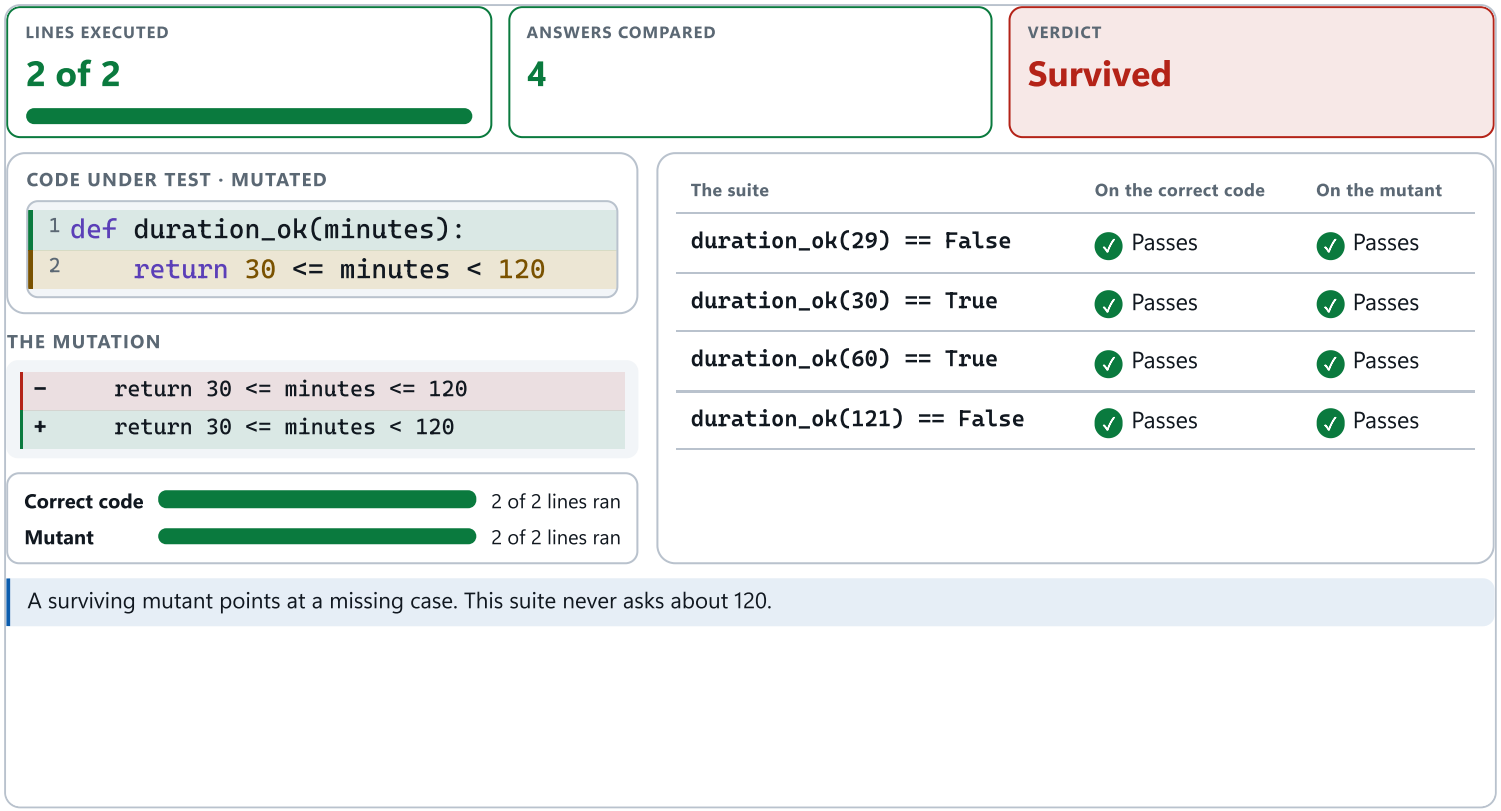
In short · Use both signals to find weak checks.

The line ran. Did anything check its answer?

Panel 2 of 4 · The mutation survives while the 120 case is missing

Condition: **Mutation <= 120 to < 120; no 120 case** State: **4 of 4**

All four checks still pass. The mutant survived. This suite has no case where the two versions answer differently.



The mutation changes <= 120 to < 120. The suite of 29, 30, 60 and 121 passes on both versions, so the mutant survives. The missing case is 120.

Principle · Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

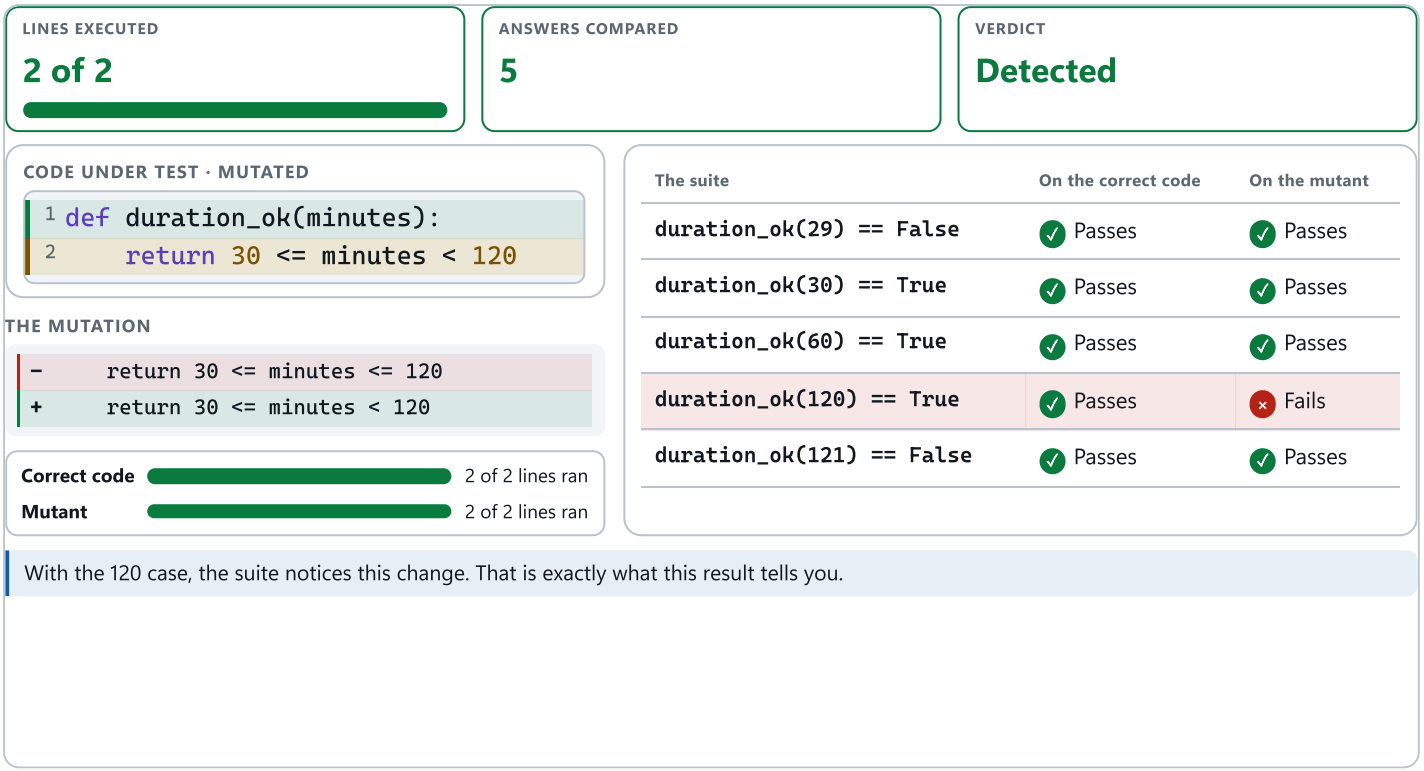
In short · Use both signals to find weak checks.

The line ran. Did anything check its answer?

Panel 3 of 4 · One added assertion detects it

Condition: **The same mutation; the suite now includes 120** State: **4 of 4**

You have learned something limited and useful: this suite notices this boundary change. R-02 still decides what is right.



Added assertion: duration_ok(120) == True. The suite now fails on the mutant. The limited conclusion: this suite notices this boundary change. R-02 still decides what is right.

Principle · Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

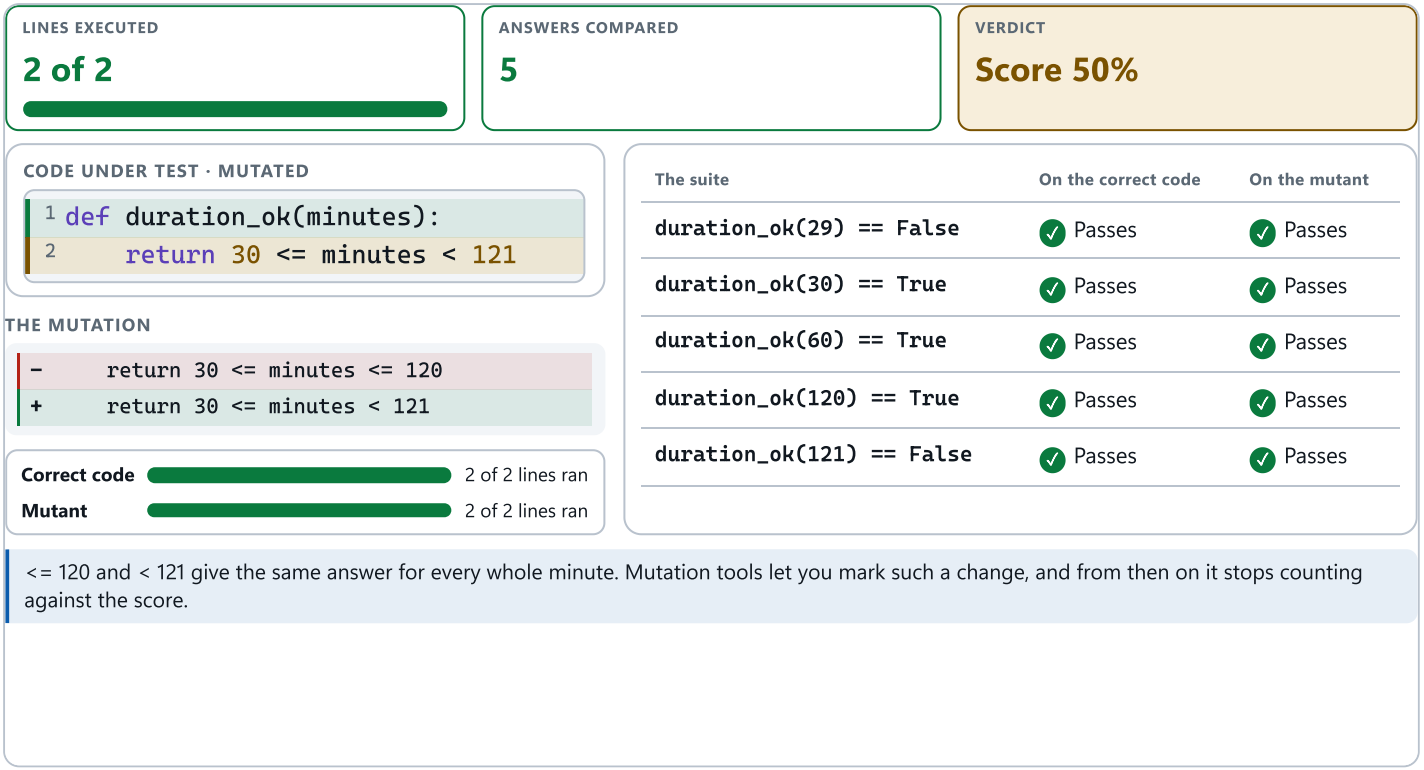
In short · Use both signals to find weak checks.

The line ran. Did anything check its answer?

Panel 4 of 4 · Equivalent changes need judgment

Condition: **A second mutation: <= 120 to < 121** State: **3 of 3**

Two mutants tried, one detected: **detected / valid = 1 / 2 = 50%**. The score points at places to look. One of these two needs no new test.



For whole minutes, <= 120 and < 121 always agree, so this mutant survives whatever you add. The score, before anything is marked, is detected / valid = 1 / 2 = 50%. Read it as a signal about where to look.

Principle · Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

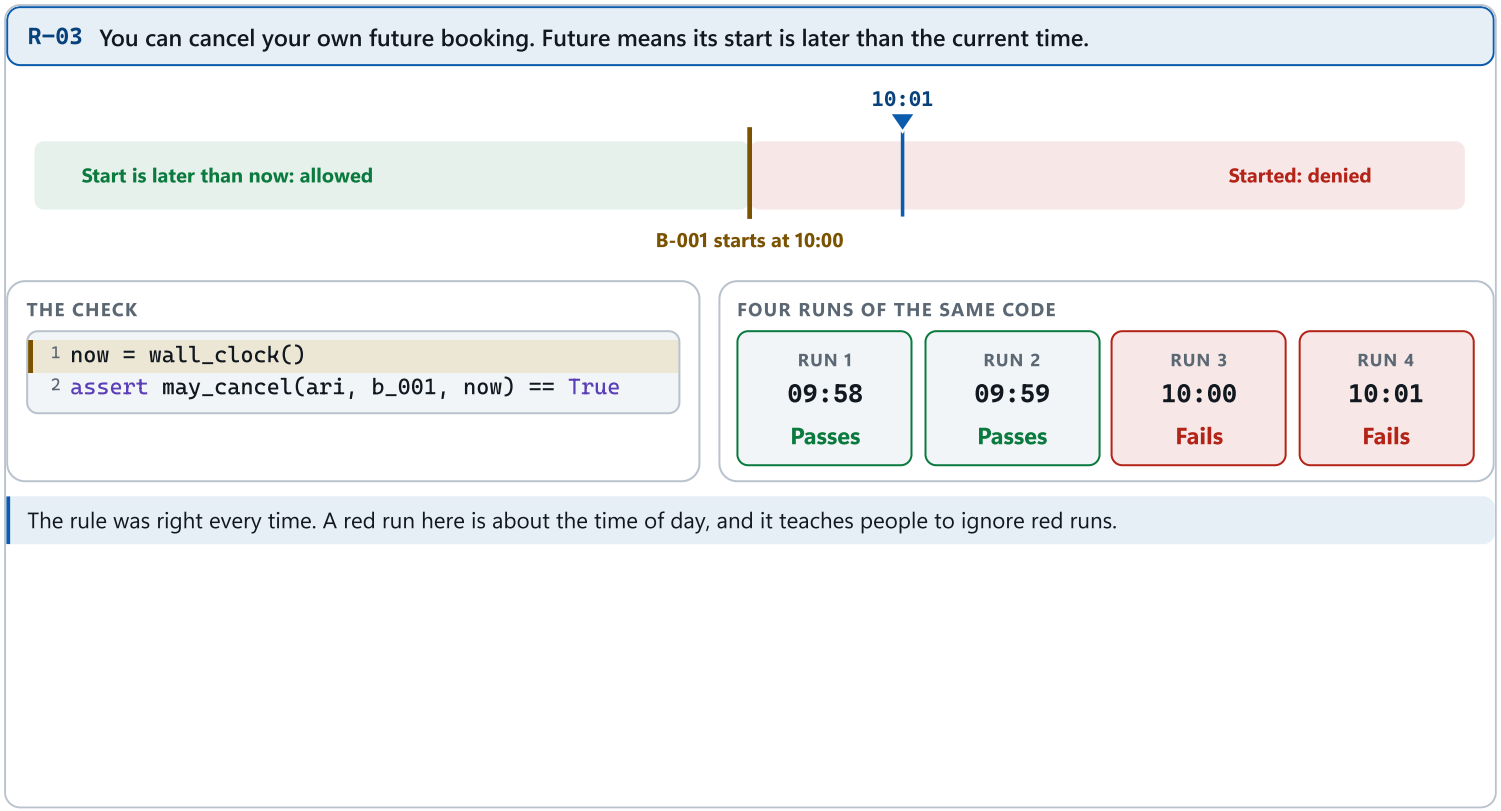
In short · Use both signals to find weak checks.

Same code, same check. Why does it pass and then fail?

Panel 1 of 3 · Uncontrolled input: the wall clock

Condition: **The check reads the wall clock** State: **5 of 5**

Run 4 at **10:01**: it fails again. **R-03 was right in all four runs.** The clock is an input this check never controlled.



The same check runs at 09:58, 09:59, 10:00 and 10:01. R-03 is right in all four runs; the clock crossed the booking's 10:00 start. These clock readings are simulated.

Principle · Control irrelevant variation so a failing check tells us something about the behavior being examined.

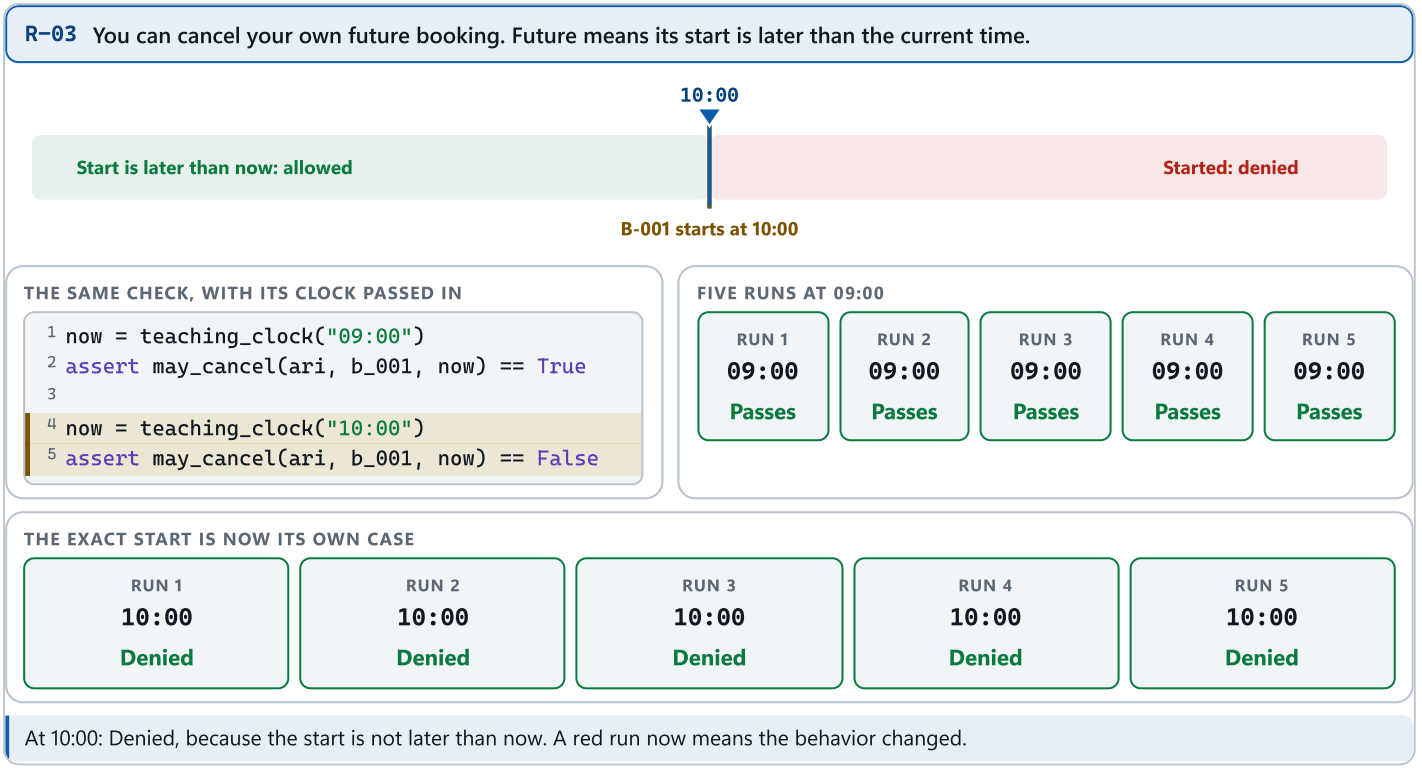
In short · Fix what should not matter, so a failure means something.

Same code, same check. Why does it pass and then fail?

Panel 2 of 3 · Controlled fixture: a clock the check sets

Condition: **The check sets its own clock** State: **3 of 3**

10:00, the exact start, is now a case of its own. Five runs, Denied every time, as R-03 says. A boundary you could only hit by luck is now a line in the check.



Fixed clock 09:00: allowed, every run. Fixed clock 10:00, the exact start: denied, every run, because the start is not later than now.

Principle · Control irrelevant variation so a failing check tells us something about the behavior being examined.

In short · Fix what should not matter, so a failure means something.

Same code, same check. Why does it pass and then fail?

Panel 3 of 3 · Changed: one injected notification failure

Condition: **The clock stays fixed; one notification failure is injected** State: **4 of 4**

Repair the handler. **Five runs, five passes:** booking Confirmed; notification Failed, visible, and ready to retry.

R-04 A failed notification leaves a confirmed booking confirmed.

THE CHECK THAT WAITS

```
1 send_notification(booking)
2 wait(2)   # seconds, and then hope
3 assert notification_arrived()
```

RUN 1	RUN 2	RUN 3	RUN 4	RUN 5
1.2 s	2.6 s	1.7 s	3.1 s	0.9 s
Passes	Fails	Passes	Fails	Passes

Same code, different delays. This red run says the message was late, and nothing about the rule.

THE INJECTED FAILURE

```
1 provider = FakeProvider(fail_next=True)
2 book(request, provider)
3 assert booking.state == "Confirmed"
4 assert notification.state == "Failed"
```

RUN 1	RUN 2	RUN 3	RUN 4	RUN 5
Confirmed	Confirmed	Confirmed	Confirmed	Confirmed
Passes	Passes	Passes	Passes	Passes

Booking Confirmed. Notification Failed, visible, and ready to retry.

The waiting check passes or fails with a simulated delay. The controlled failure repeats until the handler is repaired, and its message names R-04. After the repair: booking Confirmed; notification Failed, visible, and ready to retry.

Principle · Control irrelevant variation so a failing check tells us something about the behavior being examined.

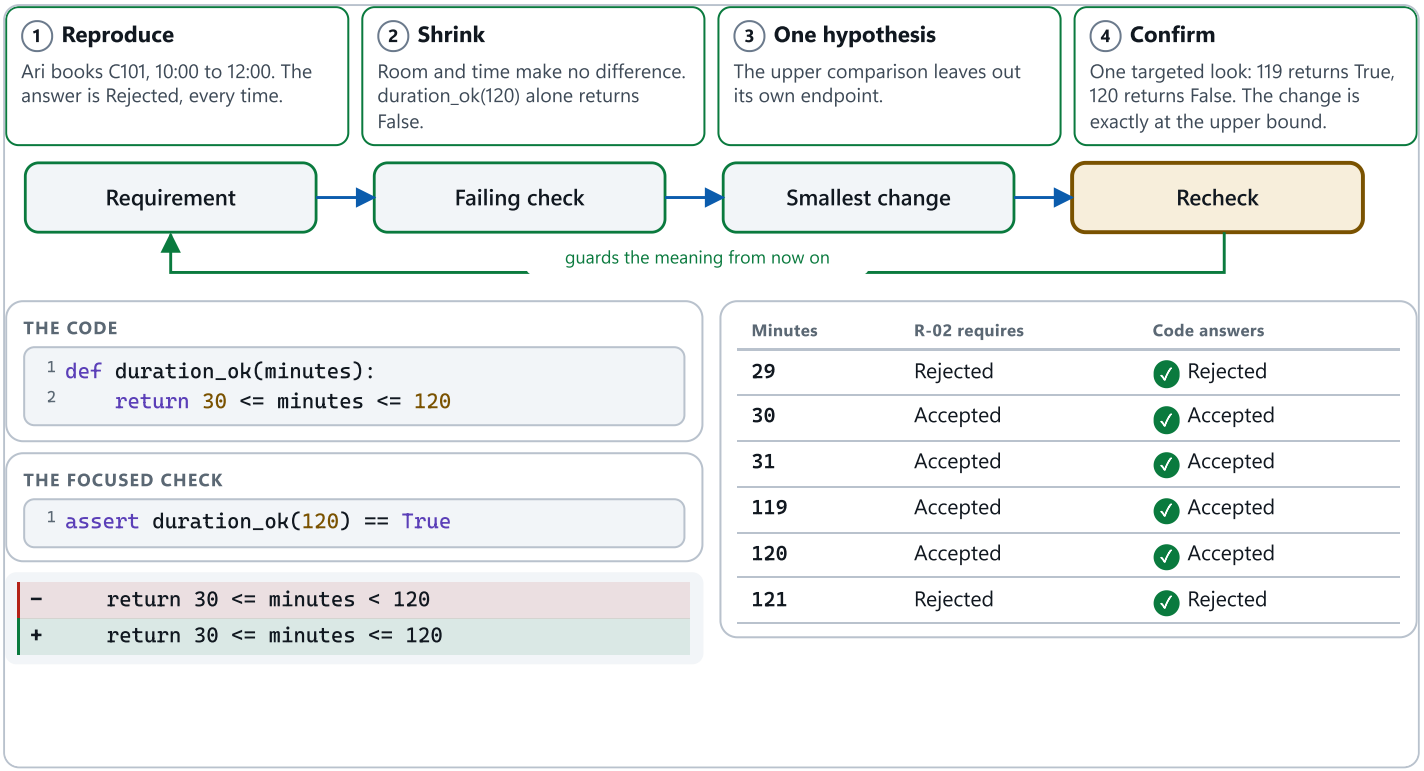
In short · Fix what should not matter, so a failure means something.

What should happen before you fix the bug?

Panel 1 of 3 · Four debugging moves and the feedback loop

Condition: **Debug in four moves, then correct under a failing check** State: **8 of 8**

Rerun. All six boundary cases pass, and this check stays to guard the meaning of the rule.



Reproduce with the 120-minute case, shrink to duration_ok(120), state one hypothesis about the comparison, and confirm with one targeted observation. Then a focused failing check, the smallest change, and a rerun of all six cases.

Principle · Tests are feedback instruments whose value depends on the claims, observations, and assumptions they preserve.

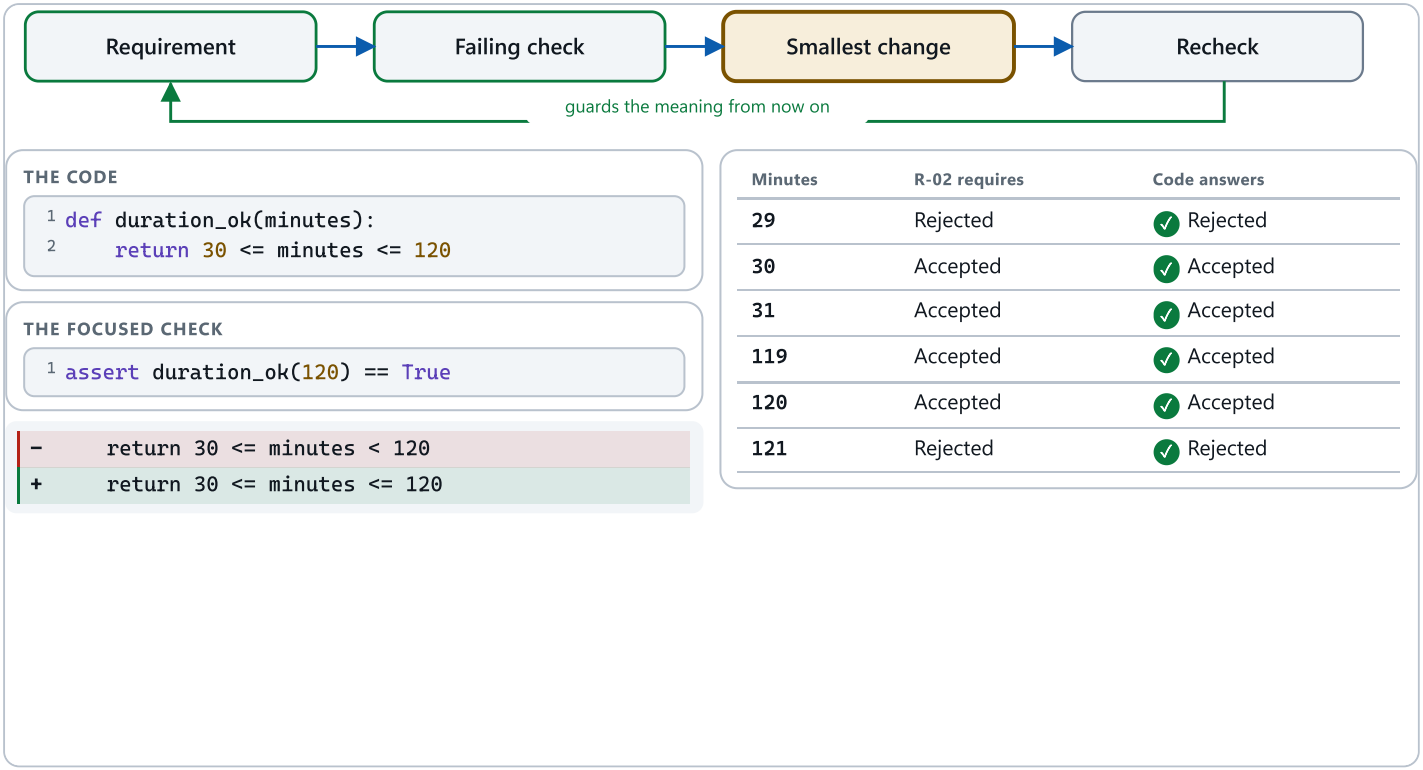
In short · A check is only as good as what it holds on to.

What should happen before you fix the bug?

Panel 2 of 3 · Changed: the check comes first

Condition: **The check is written before the code** State: **4 of 4**

The check is as right as the requirement it came from. If the requirement itself said “under 120”, a check written first would carry that mistake faithfully.



Test-first development makes the expected behavior explicit before the change. A check written from a wrong requirement carries that mistake faithfully, so the requirement still has to be right.

Principle · Tests are feedback instruments whose value depends on the claims, observations, and assumptions they preserve.

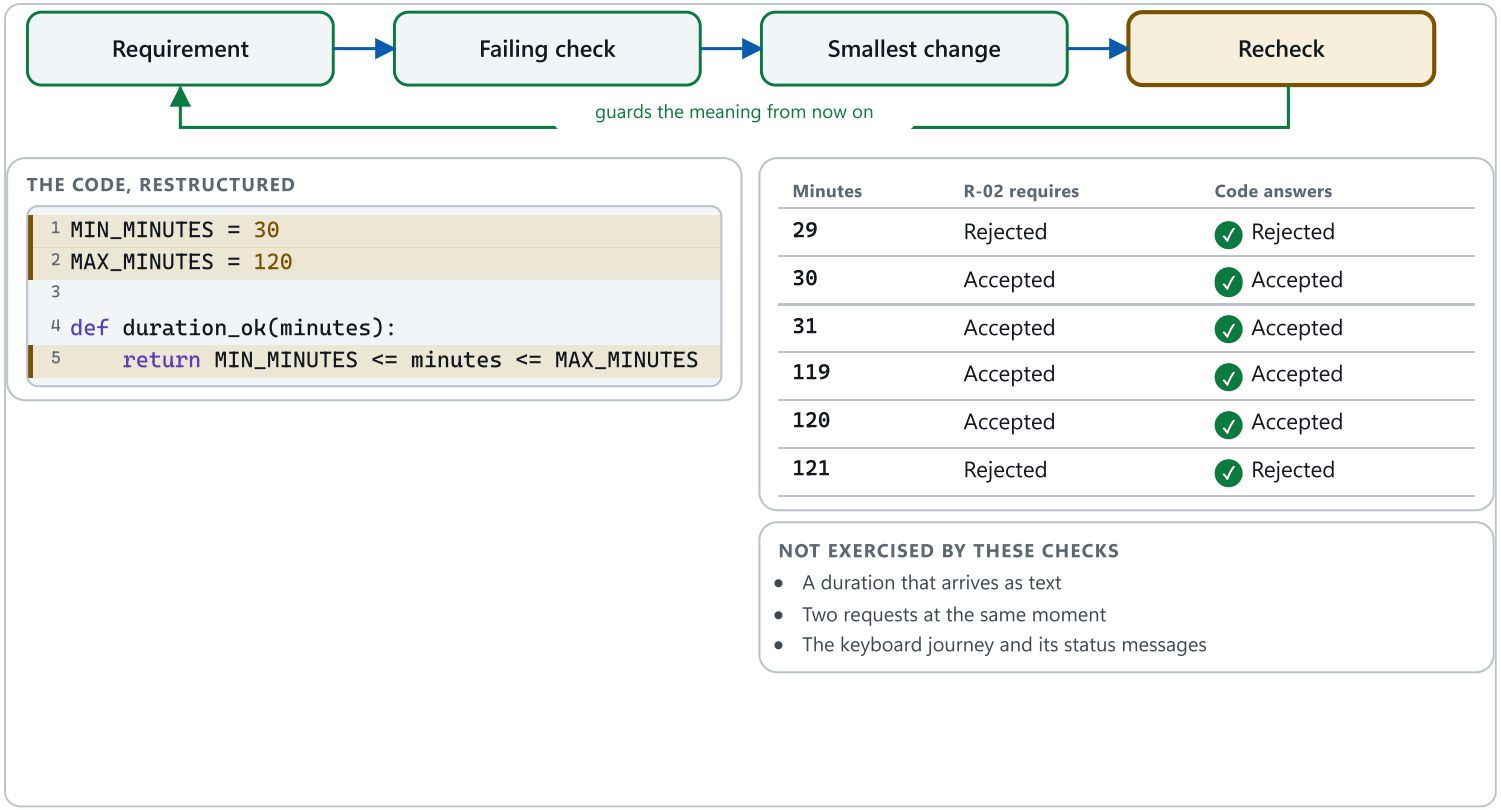
In short · A check is only as good as what it holds on to.

What should happen before you fix the bug?

Panel 3 of 3 · Changed: refactor under the checks

Condition: **Refactor the corrected predicate; its contract stays the same** State: **4 of 4**

Say what this leaves open as well: the conditions below were never exercised here. Higher-level checks still have their own job.



Named limits leave every answer unchanged. The list shows conditions these checks never exercised; higher-level checks have their own job.

Principle · Tests are feedback instruments whose value depends on the claims, observations, and assumptions they preserve.

In short · A check is only as good as what it holds on to.

What does each kind of evidence let you claim?

Reading, analyzing and running answer different questions. Keep each claim with its evidence.

Before and during execution

Review · Before running
A person compares < with "both included" in R-02. The operator contradicts the rule.

Static analysis · Before running
A tool reads every path without running the service. request.get("minutes") can be None at this comparison.

Dynamic check · While running
One execution with one input. duration_ok(120) returns False.

A contract still answers to the requirement

- Precondition: minutes is a whole number. Justified by R-02.
- Postcondition: True exactly when 30 <= minutes <= 120. Justified by R-02.
- Invariant: No two confirmed bookings for one room overlap. Justified by R-01.

Claim, evidence, remaining uncertainty

Claim	Evidence	What stays open
120 is accepted and 121 is rejected.	The boundary table and a detected boundary mutant.	Parsing the value at the request boundary.
Concurrent requests cannot confirm an overlap.	A concurrent check at the storage boundary.	Interleavings the check did not exercise.
A failed notification leaves the booking confirmed.	A controlled double and a contract check against the provider.	A later provider version, until the contract check runs again.
Reopening after a lost response shows Confirmed.	The regression example found by exploration.	Whether people understand the status. That is a separate study.

Eight principles, six checks

Decide your answer, then open each check. The final question applies the same reasoning outside room booking.

Principles

1

Test cases should distinguish plausible incorrect behavior from the intended behavior.

In short: Pick the case that can tell right from nearly right.

`week-08.html#/friendly-example`

2

Partitions organize expectations; boundaries expose where one expectation becomes another.

In short: Group the same answers, then test the edges.

`week-08.html#/partition-the-inputs`

3

The appropriate test scope follows the claim and failure risk, not the prestige of a test label.

In short: Choose the check by what you need to know.

`week-08.html#/choose-test-scope`

4

A test oracle—the basis for the expected result—must not simply repeat the defect being examined.

In short: Get the expected answer from the requirement, not from the code.

`week-08.html#/oracle-independence`

5

Properties expose classes of behavior, but the properties themselves must express the intended rule.

In short: State what must always hold, and check the statement too.

`week-08.html#/properties-and-counterexamples`

6

Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

In short: Use both signals to find weak checks.

`week-08.html#/coverage-and-mutation`

7

Control irrelevant variation so a failing check tells us something about the behavior being examined.

In short: Fix what should not matter, so a failure means something.

`week-08.html#/repeatable-observation`

8

Tests are feedback instruments whose value depends on the claims, observations, and assumptions they preserve.

In short: A check is only as good as what it holds on to.

`week-08.html#/testing-as-feedback`

Checks: decide before reading the answer

1. Which two cases distinguish an inclusive 120-minute limit from no upper limit?

> 120 must be accepted and 121 rejected, under otherwise valid conditions.

2. Do adjacent intervals overlap under the course rule?

> No. The end of one may equal the start of the next because the intervals are half-open.

3. Why test the storage boundary as well as the overlap function?

> Correct local arithmetic alone does not protect the invariant against concurrent operations.

4. Why can implementation and test agree while both are wrong?

> They may share the same mistaken assumption or calculate the expected result using the defective logic.

5. Does a controlled test double establish that the real provider honors its contract?

> No. It isolates a claim; evidence at the real boundary is still needed.

6. A file converter passes a test whose expected output is produced by the same conversion function. What would strengthen the test?

> Use an independently derived small example and a property justified by the format's meaning, including a relevant boundary. If round-trip conversion is promised, check it too; a shared defect can make both directions agree while still producing the wrong result.

The checks challenge the behavior. What if the input cannot be trusted?

Week 9 asks what is permitted, which data may cross a boundary, and what happens when a dependency fails.

Week 9 starts here

What must remain true when a request, dependency, or assumption cannot be trusted?

What carries forward

- Expected results derived from the requirement, independent of the code.
- Explicit invariants such as R-01, written as properties you can check.
- Controlled clocks and doubles, so a red check means something.

In practice · examples from 2026

pytest and JUnit run example checks. Hypothesis generates property-based cases. Stryker and mutmut run mutation analysis. coverage.py measures executed lines. Playwright drives complete browser journeys. Pact checks a consumer's expectations against a provider.

Sources

S01 IEEE Computer Society · SWEBOK https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI · CS2023 final report https://csed.acm.org/final-report/

S11 Hypothesis · property-based testing https://hypothesis.readthedocs.io/en/latest/

S12 Stryker · mutant states and metrics https://stryker-mutator.io/docs/mutation-testing-elements/mutant-states-and-metrics/

S40 Software Engineering at Google · testing overview https://abseil.io/resources/swe-book/html/ch11.html

S60 ISTQB Foundation syllabus · exploratory testing https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

S64 Pact · contract testing https://docs.pact.io/

Terms used this week

Test case
One input with one expected result.

Boundary case
A value at the edge where the expected answer changes.

Partition
A group of inputs that share one expected answer.

Assertion
A statement a check compares with the actual result.

Unit check
A check of one small piece, such as a pure function.

Integration check
A check of real parts working together.

End-to-end check
A check of a complete user journey.

Test double
A controlled stand-in for a real dependency.

Contract check
A check of one side against what the other side expects.

Exploratory testing
Choosing the next check from what you just observed.

Regression check
A check that keeps a fixed defect from coming back.

Test oracle
The basis for an expected result.

Property
A statement that should hold for many inputs.

Counterexample
An input that shows a property failing.

Shrinking
Reducing a failing input to a smaller one that still fails.

Fuzzing
Feeding generated input to find crashes and odd behavior.

Coverage
A measure of which code ran during the checks.

Mutation
A deliberate change to see whether the checks notice.

Flaky check
A check whose result changes with no change in the code.

Test-first
Writing the check before the code it checks.